



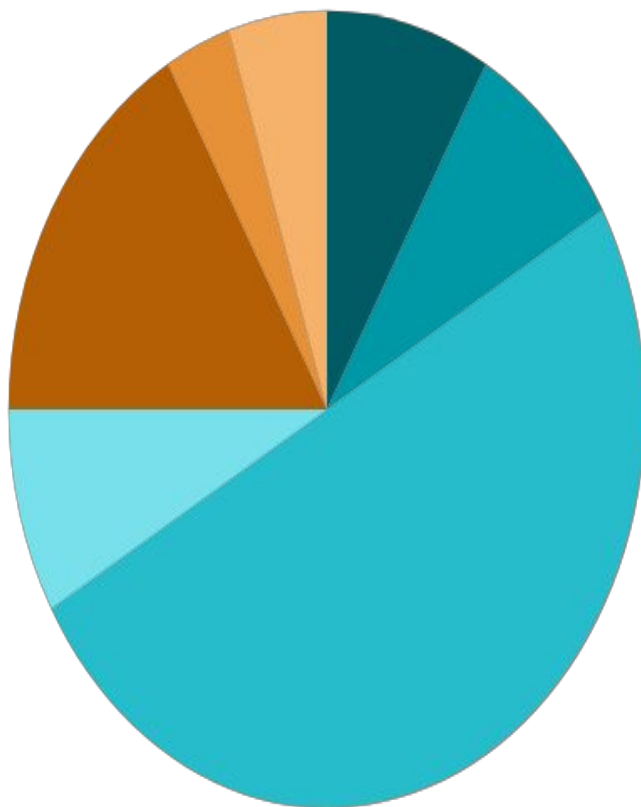
Hadoop



Outline

- Data format
- analyzing data with Hadoop
- scaling out
- Hadoop streaming
- Hadoop pipes
- Design of Hadoop distributed file system (HDFS)

Lecture Structure



**05
Minutes**

Outline & Objectives



**05
Minutes**

Recap of Previous Lecture



**30
Minutes**

**Delivery of Actual Content as per
Lesson Plan**



**05
Minutes**

Industry Relevance



**10
Minutes**

Interactive Activities



**02
Minutes**

Importance of Discipline



**03
Minutes**

Recording of Attendance

Hadoop

What is Hadoop?

- Open-source framework by Apache.
- Stores, processes, and analyzes large-scale data.
- Written in Java, used for batch/offline processing (not OLAP).
- Used by major companies like Facebook, Google, Yahoo, Twitter, LinkedIn.
- Scalable by adding nodes to the cluster.

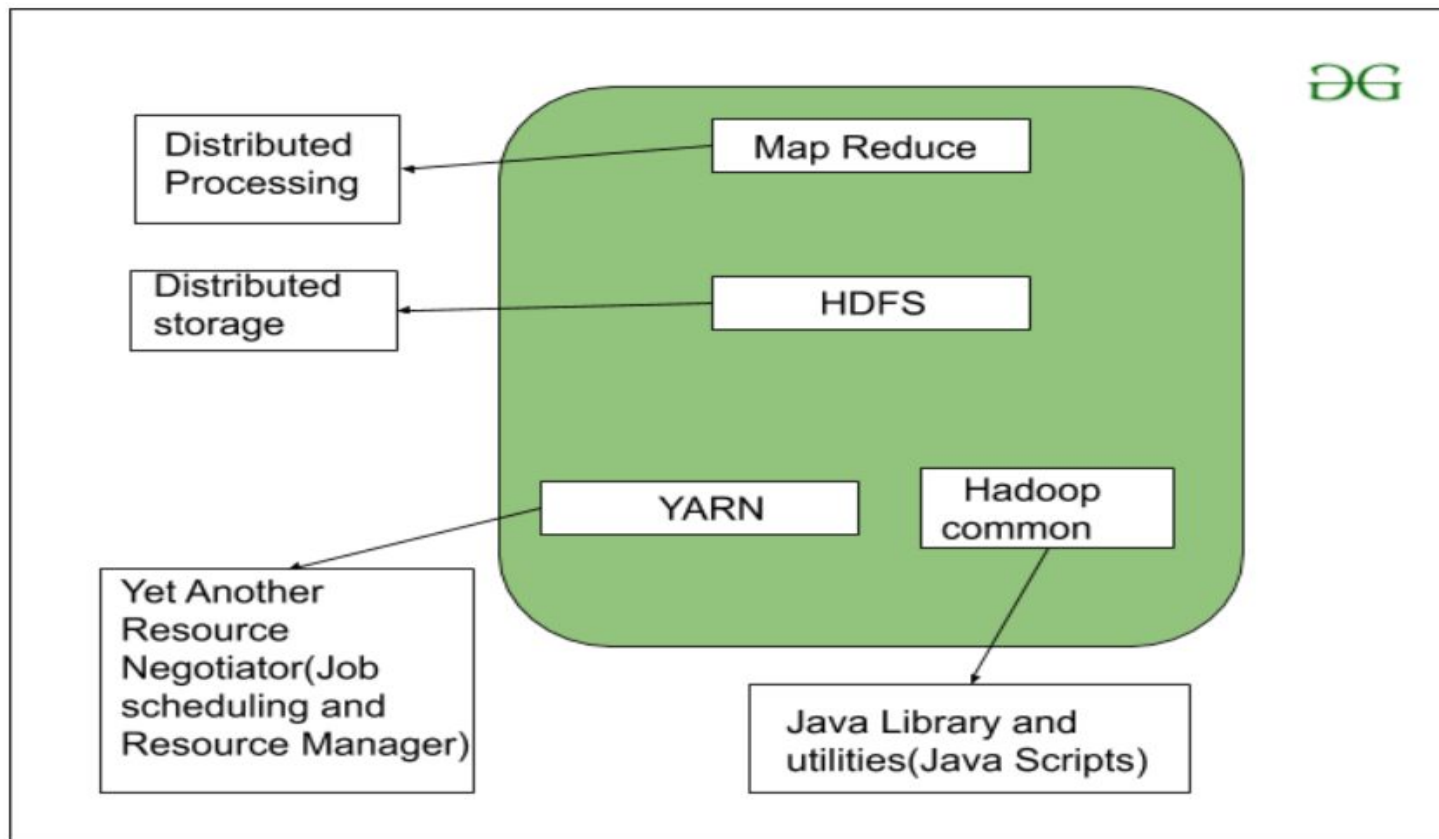
Advantages of Hadoop

- **Fast:**
 - Distributed data leads to faster retrieval and processing.
 - Processes terabytes in minutes, petabytes in hours.
- **Scalable:**
 - Can scale by simply adding more nodes.
- **Cost-effective:**
 - Open-source, uses commodity hardware.
- **Resilient to Failure:**
 - Data replication ensures reliability in case of node failure (default 3x replication).

History of Hadoop

- **2002: Apache Nutch** project started by Doug Cutting and Mike Cafarella to solve big data storage challenges.
- **2003: Google** introduces **GFS (Google File System)**, inspiring Hadoop's HDFS.
- **2004:** Google publishes the **MapReduce** paper, influencing Hadoop's MapReduce framework.
- **2005: NDFS (Nutch Distributed File System)** with MapReduce, the precursor to Hadoop.
- **2006: Hadoop** project launched by Doug Cutting at **Yahoo!**, first version 0.1.0 released.
- **2007: Yahoo!** runs large Hadoop clusters (1,000 nodes).
- **2008:** Hadoop sorts **1TB of data** in **209 seconds** on a 900-node cluster.
- **2011:** Hadoop gains mainstream adoption with major tech companies.
- **2013: Hadoop 2.0** introduces **YARN**, improving resource management and scalability.
- **2017: Hadoop 3.0** released with **Erasure Coding** for efficient data storage and support for larger clusters.
- **2020:** Hadoop faces competition from **Apache Spark** and cloud-native tools.
- **2024: Hadoop** continues to be a key part of **hybrid cloud architectures**, mainly for **batch processing** and **legacy systems**.

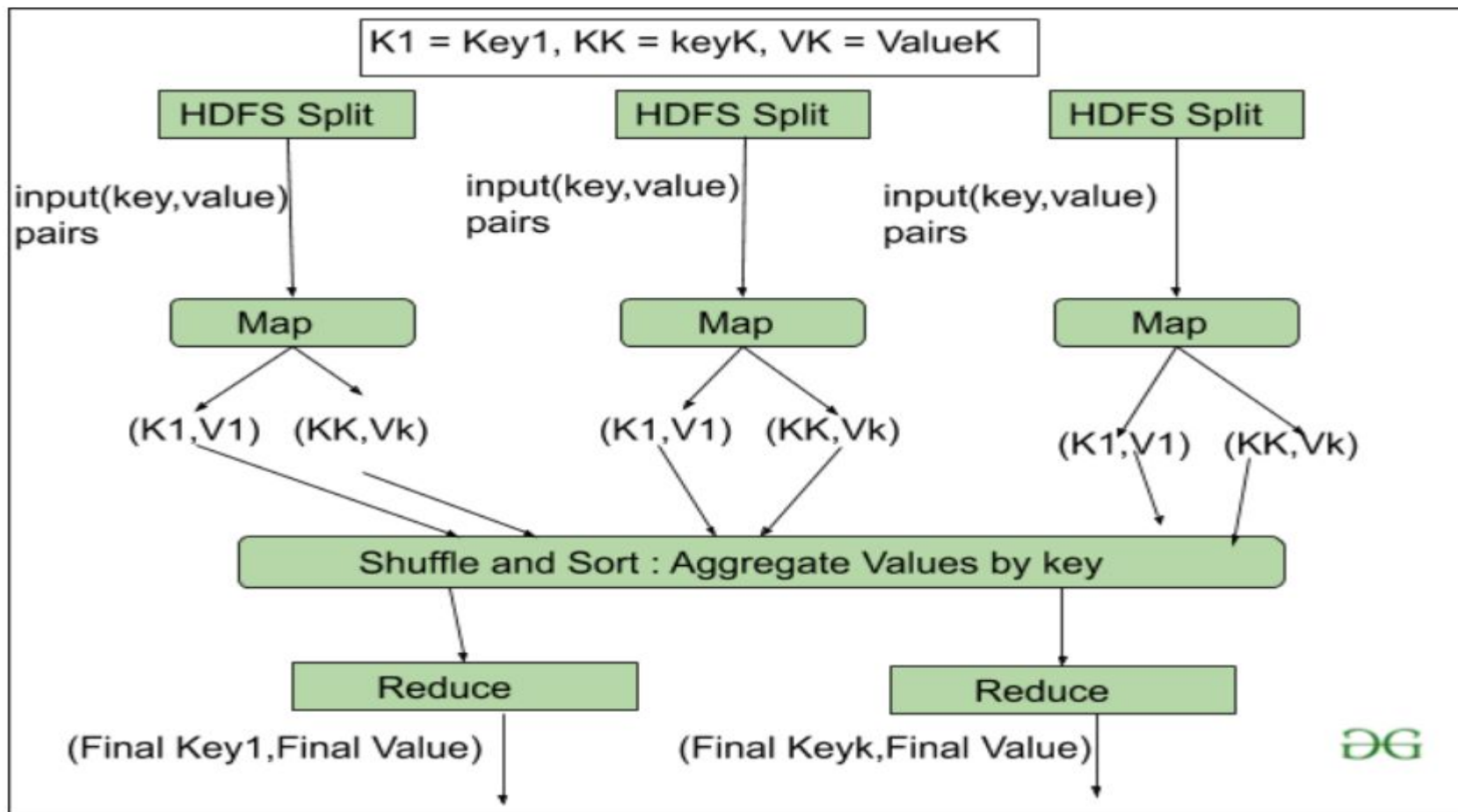
Hadoop Modules



Hadoop Modules

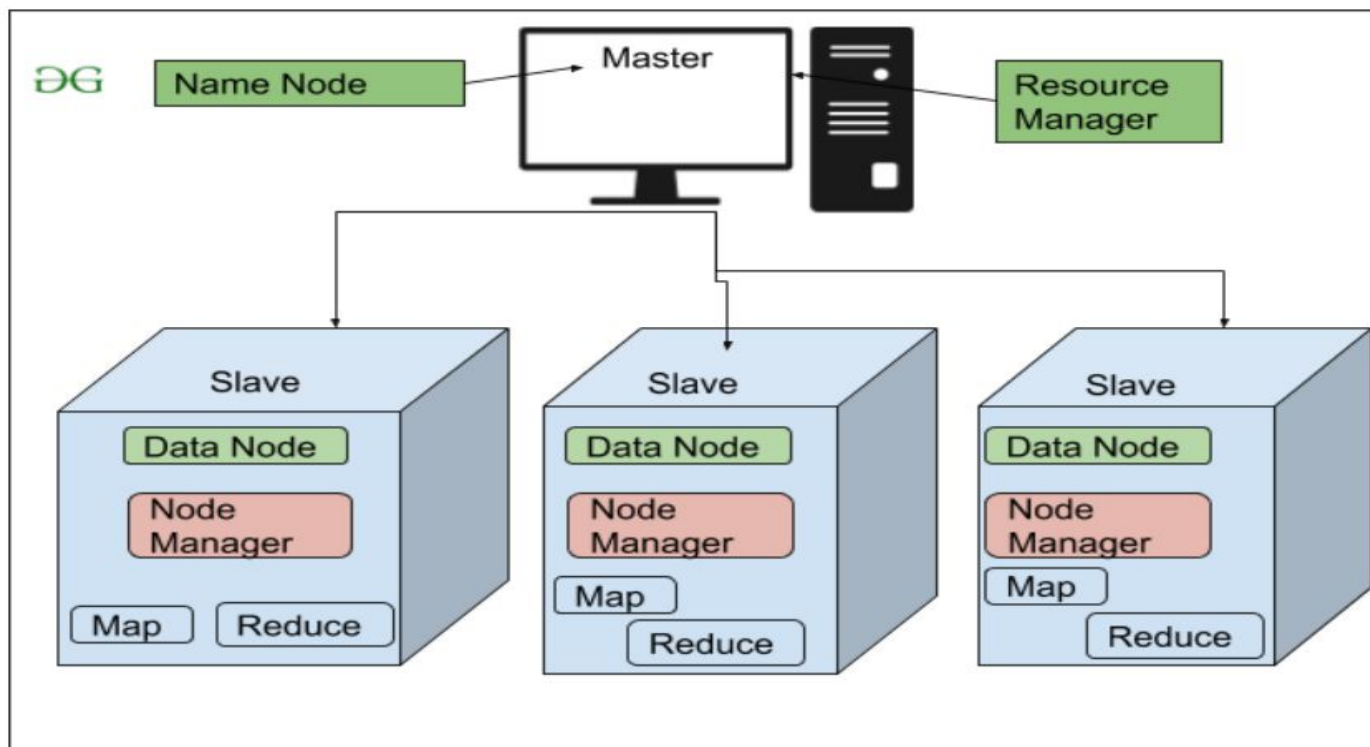
- **HDFS (Hadoop Distributed File System)**
 - Based on Google's GFS (Google File System).
 - Files are broken into blocks and stored across nodes in a distributed architecture.
- **YARN (Yet Another Resource Negotiator)**
 - Manages job scheduling and cluster resources.
- **MapReduce**
 - Java framework for parallel computation using key-value pairs.
 - Tasks: Map (convert data to key-value pairs) → Reduce (final computation).
- **Hadoop Common**
 - Java libraries to start and manage Hadoop components.

Map-Reduce





HDFS



Hadoop Architecture

- **Master Node:**
 - **Job Tracker:** Manages MapReduce job submissions.
 - **NameNode:** Manages the file system namespace (open, rename, close files).
- **Slave Node:**
 - **DataNode:** Stores data blocks and handles read/write requests.
 - **TaskTracker:** Executes tasks sent by Job Tracker (acts as Mapper).



HDFS (Hadoop Distributed File System)

- **NameNode:**
 - Single master node managing the namespace.
 - Can be a single point of failure.
- **DataNode:**
 - Multiple slave nodes storing data blocks.
 - Handles data operations like reading, writing, replication.



YARN (Yet Another Resource Negotiator)

- **YARN:** Resource management and job scheduling framework in Hadoop.
 - **Job Scheduler:** Divides tasks, tracks job dependencies, and prioritizes jobs.
 - **Resource Manager:** Allocates resources for tasks in the cluster.
- **Features:**
 - Multi-Tenancy
 - Scalability
 - Cluster Utilization

Hadoop Data Formats

- **Text/CSV Files**
 - Simple and widely used.
 - No metadata stored.
 - Block compression not supported, causing performance issues.
 - Avoid including headers to prevent null values in processing.
- **JSON Records**
 - Each line represents a JSON datum.
 - Supports metadata, but no block compression.
 - Hadoop has limited native support; third-party tools help.
- **Avro Files**
 - Stores metadata with data.
 - Supports schema evolution (renaming, adding, deleting fields).
 - Splittable and supports block compression.
 - Mature support in Hadoop ecosystem.

Hadoop Data Formats

- **Sequence Files**
 - Stores data in binary format.
 - No metadata, only supports field addition.
 - Supports block compression, often used for intermediate storage.
- **RC Files (Record Columnar Files)**
 - Columnar format for better compression and query performance.
 - No schema evolution; rewriting is required to add data.
- **ORC Files (Optimized RC Files)**
 - Improved compression and query performance over RC files.
 - Schema evolution not supported.
- **Parquet Files**
 - Columnar format with good compression and query benefits.
 - Supports schema evolution (new columns at the end).
 - Slower write performance; must handle lowercase column names in Hive.

Analyzing Data with Hadoop

Key Components of Hadoop:

- **HDFS** (Hadoop Distributed File System):
 - Distributed storage across multiple machines.
- **MapReduce**:
 - Programming model for large-scale distributed data processing.
 - **Map Phase**: Breaks data into chunks, processed in parallel by mappers.
 - **Reduce Phase**: Aggregates the data, processed by reducers.

Additional Tools for Data Analysis:

- **Apache Hive**:
 - SQL-like interface to query data in HDFS.
 - Translates SQL queries into MapReduce jobs.
- **Apache Pig**:
 - High-level scripting language for writing data processing pipelines.
 - Simplifies data transformation.
- **Apache Spark**:
 - Distributed computing framework for fast and flexible data processing.
 - Supports SQL, machine learning, and graph processing.

Analyzing Data with Hadoop

1. Data Ingestion

- Load data from various sources (files, databases, streaming platforms).
- Supports **batch processing**.

2. Data Storage

- Store data in **HDFS** (Hadoop Distributed File System) for fault tolerance and replication.
- Supports **HBase**, **Cassandra**, and cloud storage.

3. Data Processing

- **MapReduce** model: Data processed in parallel across cluster nodes.
 - **Map Phase**: Data transformed into key-value pairs.
 - **Reduce Phase**: Data aggregated to generate final results.

Analyzing Data with Hadoop

4. MapReduce Framework

- Write programs in **Java**, **Python**, or other languages.
- Handles task distribution, fault tolerance, and scalability.

5. Querying & Analysis

- Tools for querying:
 - **Apache Hive** (SQL-like interface).
 - **Apache Pig** (high-level scripting).
 - **Apache Spark & Flink** (advanced analytics: ML, graph processing, real-time).

6. Data Visualization & Reporting

- Integrate with tools like **Apache Zeppelin**, **Tableau**, **Power BI** for interactive dashboards and reports.

Example

1. Data Ingestion (Loading Data into HDFS)

- We start with a text file called input.txt. The contents of input.txt could look like this:
- Text File:

The quick brown fox jumps over the lazy dog the quick
brown fox jumps

Load the data into HDFS:

- `hadoop fs -put input.txt /user/hadoop/input/`

2. Map Phase (Splitting and Mapping)

For example, the Mapper will take the line of text:

The quick brown fox

And break it down into key-value pairs:

(The, 1)
(quick, 1)
(brown, 1)
(fox, 1)

Map Phase Output (Key-Value Pairs):

(the, 1)
(quick, 1)
(brown, 1)
(fox, 1)
(jumps, 1)
(over, 1) (the, 1) (lazy, 1)
(dog, 1) (the, 1) (quick, 1)
(brown, 1) (fox, 1)
(jumps, 1)



3. Shuffle and Sort Phase: Hadoop groups all the key-value pairs by key (i.e., by word) and sorts them.

Example of Sorted and Grouped Key-Value Pairs:

```
(the, [1, 1, 1])  
(quick, [1, 1])  
(brown, [1, 1])  
(fox, [1, 1])  
(jumps, [1, 1])  
(over, [1])  
(lazy, [1])  
(dog, [1])
```

4. Reduce Phase (Aggregation): Reducer Code (Example in Java):

```
public class WordCountReducer extends Reducer<Text, IntWritable, Text,
    IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```



5. Reduce Phase Output (Final Word Counts):

```
(the, 3)
(quick, 2)
(brown, 2)
(fox, 2)
(jumps, 2)
(over, 1)
(lazy, 1)
(dog, 1)
```

6. Running the Job:

```
hadoop jar wordcount.jar WordCount /user/hadoop/input /user/hadoop/output
```

- wordcount.jar is the compiled JAR file containing the mapper and reducer code.
- /user/hadoop/input is the input directory in HDFS (containing input.txt).
- /user/hadoop/output is the output directory where results will be stored.



7. Check the Output:

```
hadoop fs -cat /user/hadoop/output/part-r-00000
```

O/P:

```
(the, 3)
(quick, 2)
(brown, 2)
(fox, 2)
(jumps, 2)
(over, 1)
(lazy, 1)
(dog, 1)
```


Scaling Out vs. Scaling Up

Scaling Out (Horizontal Scaling)

- **Definition:** Adding more machines to the infrastructure to distribute the load.
- **Key Points:**
 - Increases capacity by adding more servers.
 - Allows better fault tolerance and system resilience.
 - Handles large-scale distributed systems more efficiently.

Scaling Up (Vertical Scaling)

- **Definition:** Adding resources to an existing machine (e.g., more CPU, RAM, or storage).
- **Key Points:**
 - Involves enhancing the power of a single server or node.
 - Does not require system re-architecture.
 - Ideal for short-term needs or applications with specific performance bottlenecks.

When to Scale Up

- **Performance Impact:** When workloads hit capacity limits, leading to latency or bottlenecks.
- **Storage Optimization Fails:** When optimization becomes ineffective.
- **Application Complexity:** When handling distributed systems becomes difficult.

Strengths of Scaling Up

- **Speed:** Increases system performance quickly.
- **Simplicity:** Minimal changes to the system architecture.
- **Cost-Effective:** Lower operational costs and no need for additional infrastructure.

Weaknesses of Scaling Up

- **Latency:** Higher capacity doesn't always result in better performance.
- **Labor/Risks:** Data migration and system upgrades can lead to downtime.
- **Aging Hardware:** Limited lifespan of physical components can cause performance degradation over time.

Example Scenario:

- ERP system for a large manufacturing company needs scaling up as data increases. Adding more CPU, RAM, and storage can improve performance and handle increased data volume effectively.



Scaling out

- Replaces hardware to scale functionality, performance, and capacity.
- More efficient and effective than scale-up infrastructure.
- Cloud-based scaling ensures no need to buy new hardware for upgrades.

When to Scale Out?

- **Long-Term Strategy:** Scales infrastructure for expected data growth.
- **Flexible Upgrades:** Avoids outdated tech and vendor lock-in.
- **Distributed Storage:** Ideal for workloads spread across multiple nodes.
- **Example: Streaming Services**
- Companies like Netflix or YouTube scale out by adding servers in various regions to improve content delivery and reduce latency.

Strengths & Weaknesses of Scaling Out

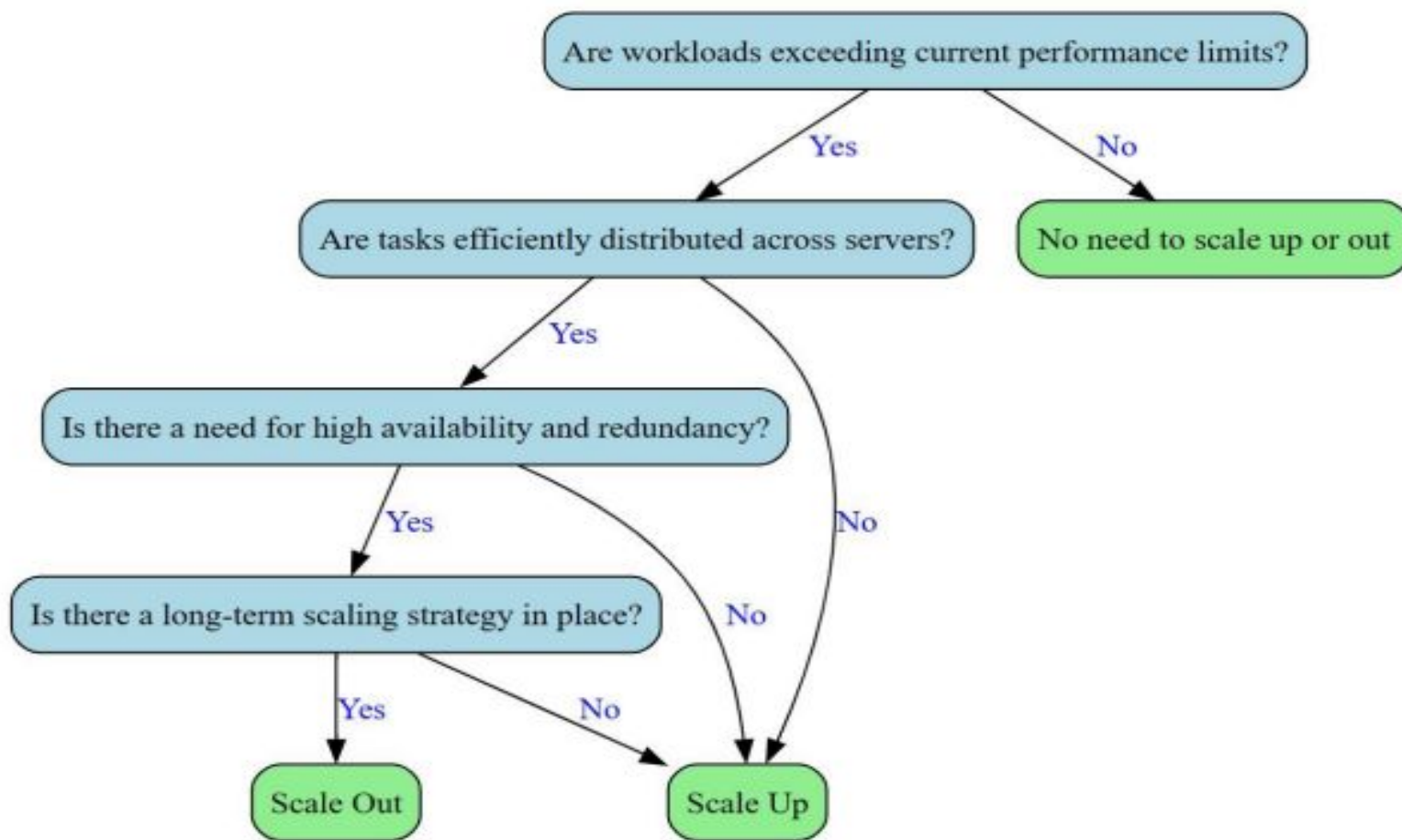
- **Strengths**

- **Newer Server Technologies:** Not constrained by older hardware, minimizing capacity and performance issues.
- **Adaptability:** Easily adjust to demand changes by adding or removing services and hardware.
- **Cost Management:** Incremental, predictable costs with pay-as-you-go model.

- **Weaknesses**

- **Limited Rack Space:** Risk of running out of space, hindering growth.
- **Increased Operational Costs:** Additional costs for power, cooling, and licensing.
- **Higher Upfront Costs:** Requires significant initial investment for setup.

Scale up or scale out? How to decide



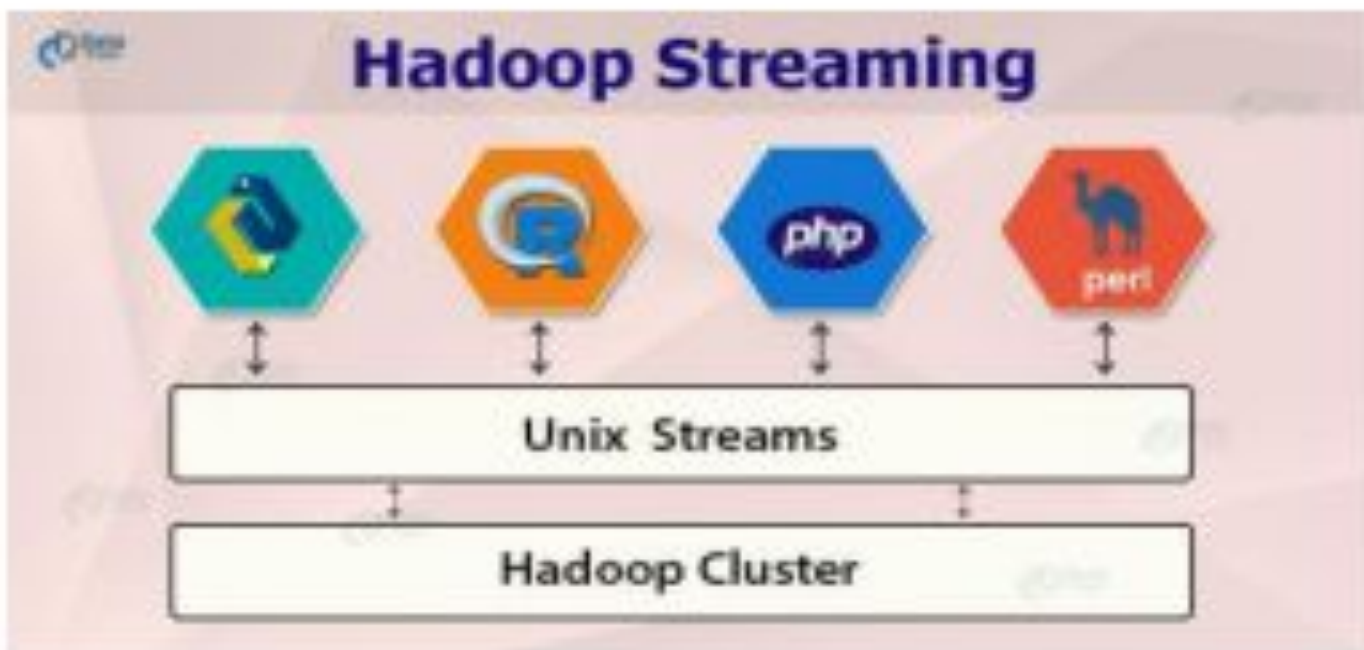


Hadoop streaming

- Utility that enables running MapReduce scripts in any language (Java or non-Java) as mapper/reducer.
- By default, Hadoop MapReduce framework is written in Java, but Hadoop provides APIs to run MapReduce programs in other languages.
- Uses Unix streams to interface between Hadoop and the MapReduce program, allowing any language that can read standard input and write to standard output.
- **Supported Languages**
Java, Python, Perl, R, PHP, C++
- **Syntax for Hadoop Streaming**

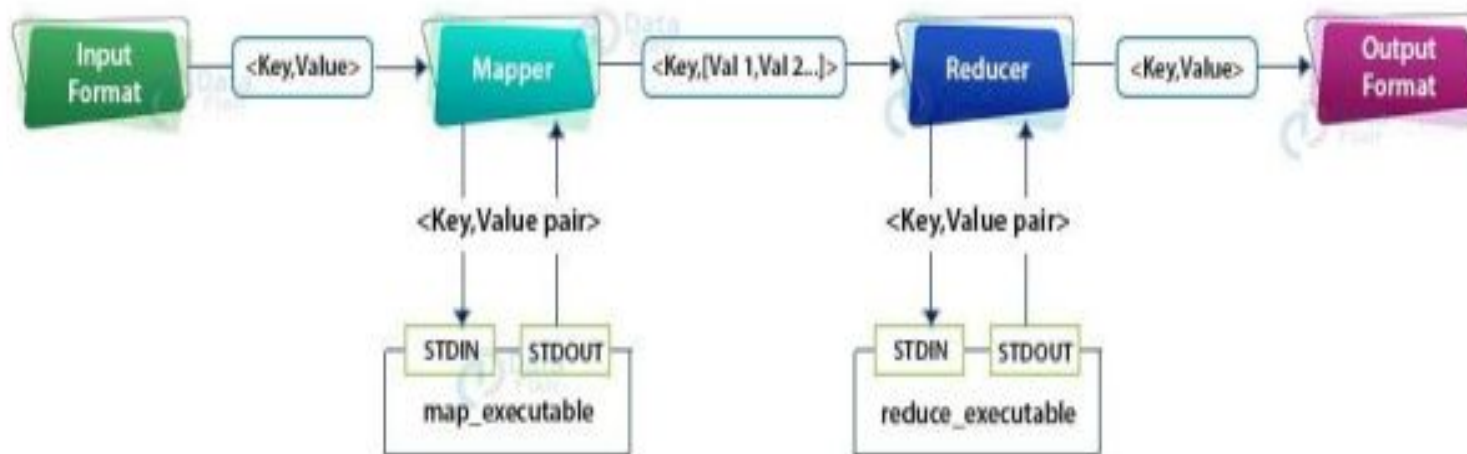
```
$HADOOP_HOME/bin/hadoop jar $HADOOP_HOME/hadoop-streaming.jar \  
-input myInputDirs \  
-output myOutputDir \  
-mapper /bin/cat \  
-reducer /usr/bin/wc
```

Hadoop streaming



- **Parameter Descriptions**
- **-input:** Input location for mapper
- **-output:** Output location for reducer
- **-mapper:** Mapper executable
- **-reducer:** Reducer executable

Hadoop Streaming



Hadoop Streaming Command Options

- **General Syntax:**
mapred streaming [genericOptions] [streamingOptions]
- **Streaming Command Options:**
- **-input directoryname or filename** (Required)
 - Specifies the input location for the mapper.
- **-output directoryname** (Required)
 - Specifies the output location for the reducer.
- **-mapper executable or JavaClassName** (Optional)
 - Specifies the Mapper executable (default: IdentityMapper).
- **-reducer executable or JavaClassName** (Optional)
 - Specifies the Reducer executable (default: IdentityReducer).
- **-inputformat JavaClassName** (Optional)
 - Specifies the input format class (default: TextInputFormat).

Hadoop Streaming Command Options

- **-outputformat JavaClassName** (Optional)
 - Specifies the output format class (default: TextOutputFormat).
- **-numReduceTasks** (Optional)
 - Specifies the number of reducers.
- **-file filename** (Optional)
 - Makes the mapper, reducer, or combiner executable available locally.
- **-mapdebug** (Optional)
 - Script called when the map task fails.
- **-reduceddebug** (Optional)
 - Script called when the reduce task fails.
- **-partitioner JavaClassName** (Optional)
 - Specifies the class to determine which reducer a key is sent to.

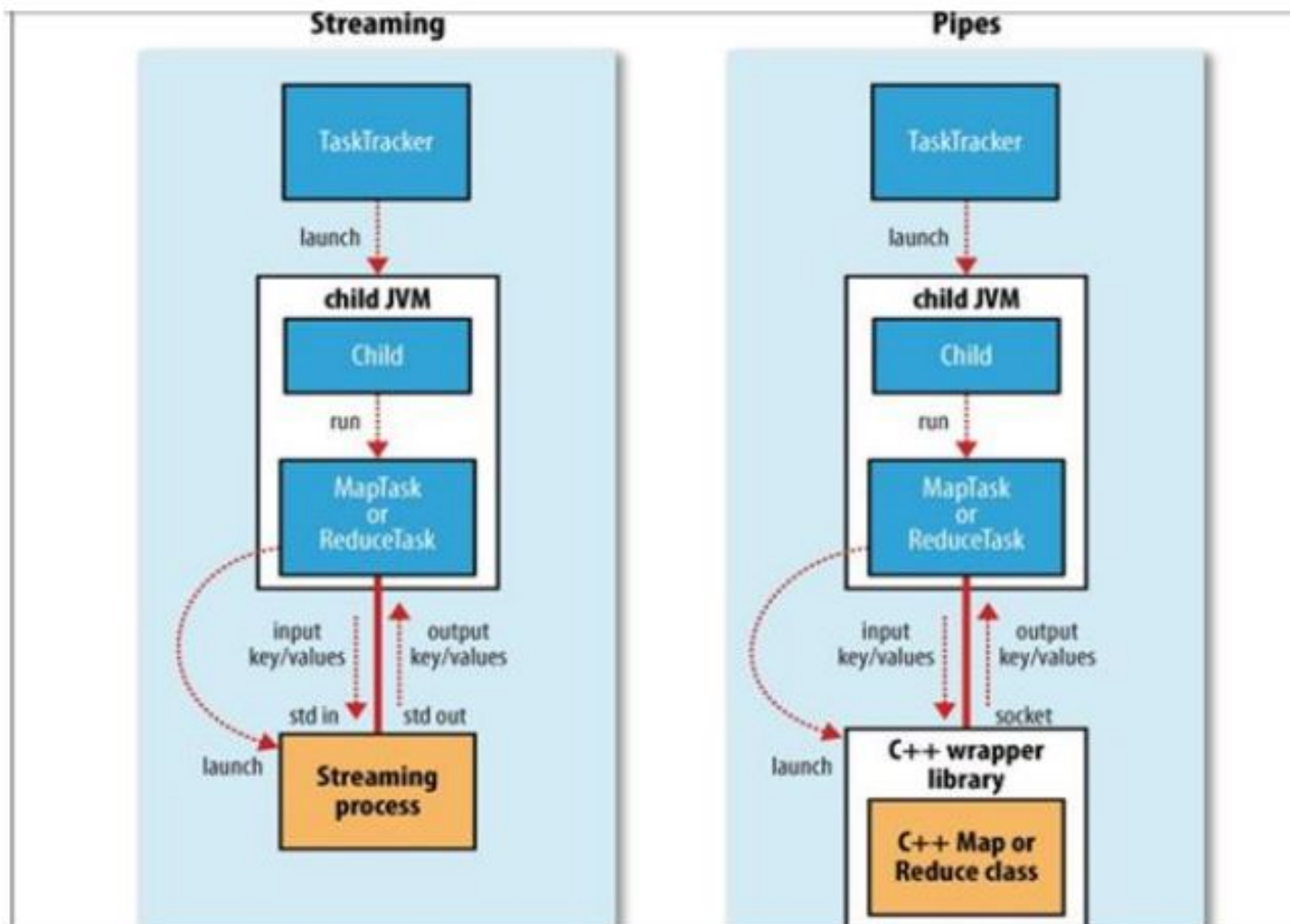
Hadoop Pipes

What is Hadoop Pipes?

- **C++ Interface** to Hadoop MapReduce
 - Allows writing MapReduce programs in C++ (or compatible languages).
 - More efficient, low-level interface than Hadoop Streaming.
 - Uses **sockets** for communication between Task Tracker and the process running the C++ code.

Key Features

- **Input & Output Format:**
 - Standard input (stdin) and output (stdout) for data.
- **Mapper & Reducer Programs:**
 - **Mapper:** Processes data, emits intermediate key-value pairs.
 - **Reducer:** Aggregates data, produces final output.





Hadoop Pipes Architecture & Benefits

Components & Execution Flow

- **API & Library:**
 - Provides C++ library to interact with Hadoop framework.
- **Compilation & Execution:**
 - Mapper/Reducer programs compiled with Hadoop Pipes library.
 - Executables submitted to the Hadoop cluster.
- **Data Processing Flow:**
 - **Input Splitting & Task Execution:** Data split into input splits, assigned to mappers.
 - **Data Streaming & Serialization:** Key-value pairs streamed between tasks.
 - **Reducer Input/Output:** Reducers process shuffled data and write final output to stdout.

Benefits

- **Efficiency:** Lower-level integration for better performance.
- **Flexibility:** Enables use of C++ or other compatible languages for Hadoop jobs.

Hadoop Streaming vs Hadoop Pipes

Feature	Hadoop Streaming	Hadoop Pipes
Languages Supported	Any language that can read from and write to the command line (e.g., Python, Ruby).	Mainly for C++, or languages that work with C++.
How It Works	Programs read data from the command line and send output back through it (stdin and stdout).	Programs use sockets to talk directly to Hadoop through a special C++ library.
Complexity	Easier to use, especially for people who write code in languages like Python.	More complex because it's a lower-level tool, usually used by C++ developers.
Performance	Can be slower because it uses text and can involve extra steps to read and write data.	Faster and more efficient because it's directly integrated with Hadoop and uses binary data.
Use Case	Great for simple jobs and quick coding in languages like Python or Ruby.	Best for performance-critical tasks that need C++ or more control over how data is processed.



Design of Hadoop Distributed File System

What is Hadoop HDFS?

- HDFS is a highly reliable and fault-tolerant storage system.
- Stores large files efficiently on a cluster of commodity hardware.
- Works on the principle of storing fewer large files rather than numerous small files

Good Practice: Storing **10 files of 100GB** each. ✓

Bad Practice: Storing **1 million files of 1MB** each. ✗.

- Ensures high throughput by enabling parallel data access.

HDFS Assumptions and Goals

1. Hardware Failure

- Hardware failures are common in large clusters.
- HDFS is designed for automatic fault detection and quick recovery.
- Ensures reliability even with frequent hardware issues.

2. Streaming Data Access

- Optimized for batch processing rather than interactive use.
- Prioritizes high throughput over low latency.
- Ensures fast data retrieval for large-scale analytics.

HDFS Assumptions and Goals

3. Large Datasets

- Handles files ranging from gigabytes to petabytes.
- Scales up efficiently to hundreds of nodes.
- Supports millions of files in a single instance.

4. Simple Coherency Model

- Follows a "write-once-read-many" model.
- Prevents data coherency issues and boosts performance.
- Ideal for applications like web crawling and MapReduce.

5. Moving Computation Closer to Data

- More efficient than moving data to computation.
- Increases system throughput and reduces network congestion.
- **Example**

❖ Imagine you have **100GB of logs** stored across different machines:

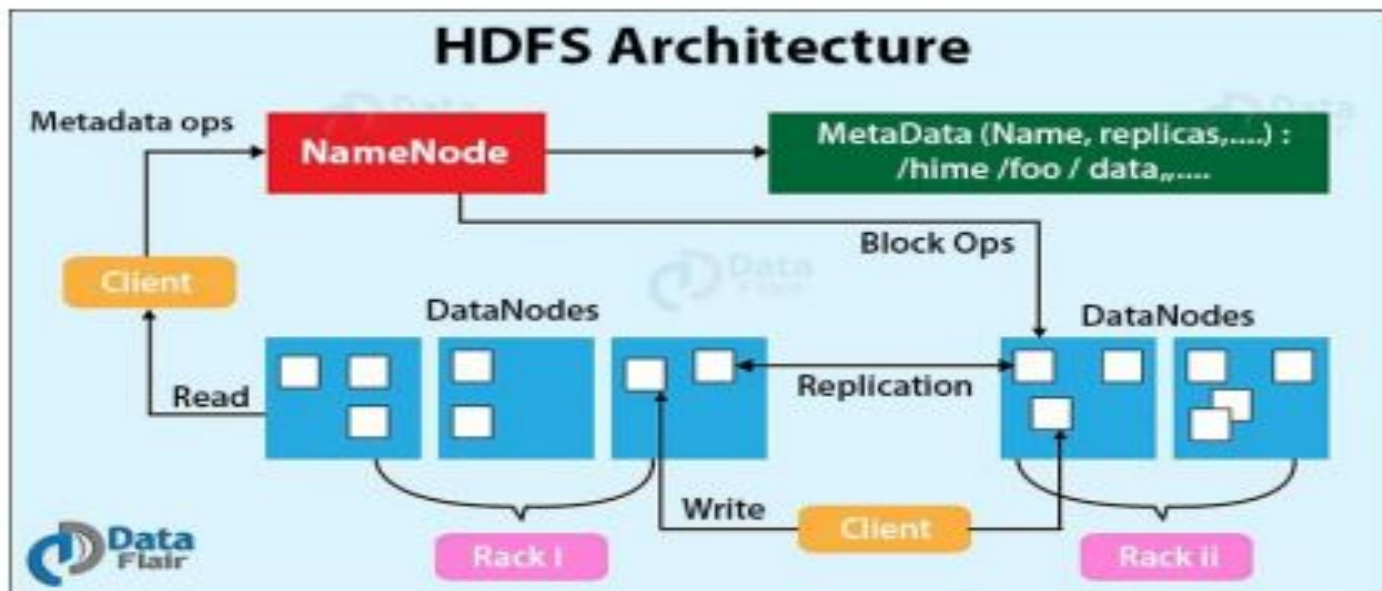
- ❖  **Bad Approach:** Copying all 100GB to a single machine for processing (very slow .
- ❖  **Good Approach:** Running a small program on each machine to process its local data (fast .

6. Portability Across Platforms

- Designed to be portable across different hardware and software platforms.
- Ensures broad adoption and scalability for large data processing needs.

HDFS Architecture

- Hadoop Distributed File System follows a Master-Slave architecture.
- A single Master Node (NameNode) and multiple Slave Nodes (DataNodes).
- Files are divided into blocks and stored on different DataNodes.
- Replication factor ensures fault tolerance.



HDFS Architecture

- **Master Node:** NameNode
- **Slave Nodes:** DataNodes
- **Additional Node:** Secondary NameNode
- NameNode manages metadata and DataNodes store file blocks.

NameNode - The Master Node

- Manages the file system namespace.
- Provides access permissions to clients.
- Stores metadata (block locations, permissions) in:
 - **Fsimage:** Complete namespace snapshot.
 - **Edit log:** Recent changes since the last Fsimage.

Functions of NameNode

- Executes file system namespace operations (open, rename, close).
- Manages and maintains DataNodes.
- Maps file blocks to DataNodes.
- Records file system changes.
- Tracks block locations.
- Ensures replication factor.
- Receives heartbeat & block reports from DataNodes.
- Reallocates blocks upon DataNode failure.

DataNodes - The Slave Nodes

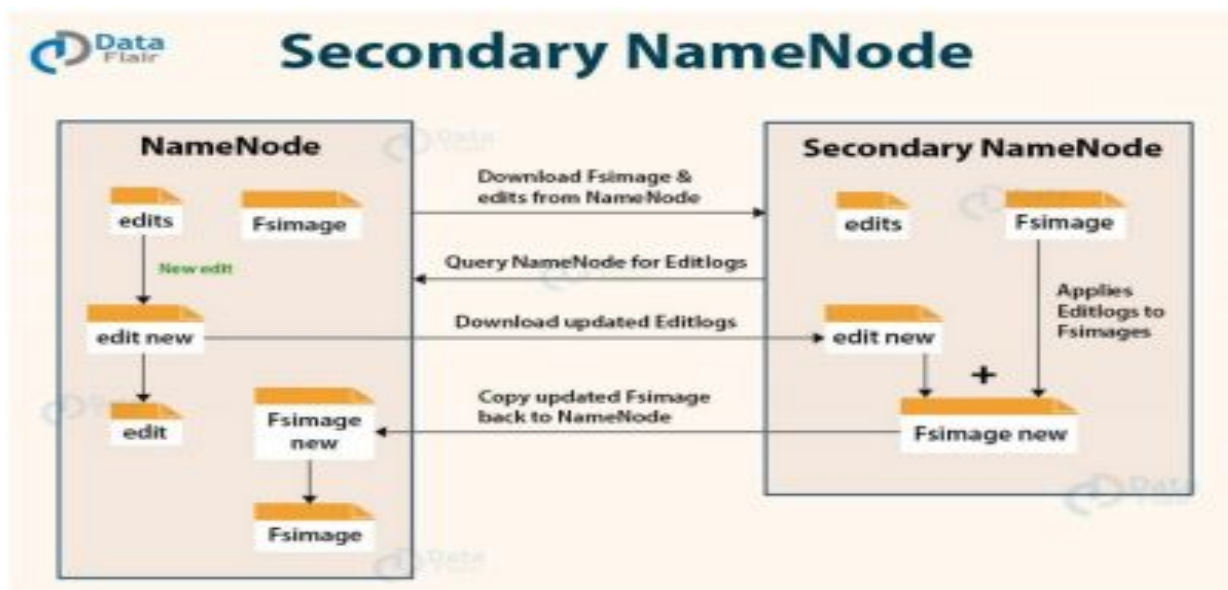
- Store file blocks.
- Use commodity hardware for cost-effectiveness.
- Serve client read/write requests.

Functions of DataNode

- Handles client read/write operations.
- Creates, replicates, and deletes blocks as per NameNode's instructions.
- Sends heartbeats to NameNode for health checks.
- Sends block reports to NameNode.

Secondary NameNode - The Helper Node

- Works alongside the primary NameNode.
- Merges **Fsimage** and **edit logs** to maintain an updated namespace.
- Helps reduce NameNode restart time.
- Can be used for metadata recovery in case of NameNode failure.



Hadoop 2 - High Availability

- Before Hadoop 2, NameNode was a single point of failure.
- Hadoop 2 introduced multiple NameNodes in hot standby mode.
- Improves reliability and fault tolerance.

Checkpoint Node

- Periodically creates checkpoints of the namespace.
- Downloads Fsimage and edits from the NameNode.
- Merges Fsimage and edits locally and uploads the new image back to NameNode.
- Stores the latest checkpoint in a structured directory

Backup Node

- Provides the same checkpointing functionality as Checkpoint Node.
- Maintains an in-memory, up-to-date copy of the namespace.
- Synchronizes with the active NameNode continuously.
- Saves the namespace into Fsimage file locally and resets edits.
- NameNode supports one Backup Node at a time.

Difference between Checkpoint node and Backup Node

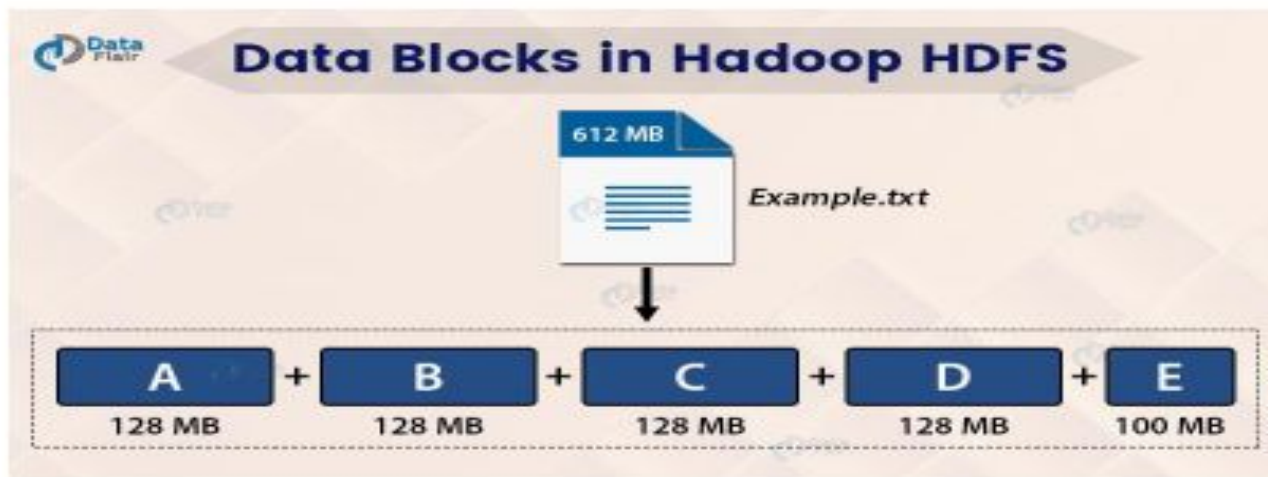
Feature	Checkpoint Node	Backup Node
Purpose	Merges FsImage and Edit Logs to create a new checkpoint for faster recovery.	Keeps an up-to-date copy of FsImage and maintains all changes in memory.
Metadata Storage	Stores a copy of the namespace but does not keep it updated in real-time.	Stores a real-time synchronized copy of the NameNode's namespace.
Synchronization	Periodically fetches edits from NameNode, applies them, and saves an updated image.	Constantly synchronized with the NameNode, receiving updates in real-time.
Failure Handling	Helps in NameNode recovery but requires extra processing to reconstruct data.	Can take over NameNode operations faster if NameNode fails.
Memory Usage	Lower, as it does not store real-time updates.	Higher, since it keeps an in-memory copy of the file system.
Function	Helps optimize NameNode performance by reducing Edit Log size.	Acts as a standby NameNode that can take over in case of failure.



- **Checkpoint Node:** reduces **edit log size** and improves **NameNode efficiency**.
- **Backup Node:** maintains a **live backup** of the NameNode's namespace.
- **Backup Node:** is **more powerful** than Checkpoint Node but requires more memory.

Blocks in HDFS

- Files are split into block-sized chunks.
- Default block size: **128MB** (configurable).
- Example: A **612MB** file is split into **four 128MB blocks** and **one 100MB block**.
- Small files occupy only required space, not full block size.



Replication Management in HDFS

- Ensures data redundancy for fault tolerance.
- Replication factor: Number of copies of each block.
- Default replication factor: **3** (can be configured).
- Example: A **128MB** block with replication factor **3** occupies **384MB** of disk space.
- Data is accessible from other DataNodes if one fails.



What is a Rack in HDFS?

- A rack consists of **40-50 DataNodes** connected to the same network switch.
- If the switch or network fails, all nodes in the rack become **unavailable**.
- HDFS uses **rack-aware placement** for data replication.
- **Rack Awareness** helps in intelligent data placement across different racks.
- Ensures that replicas are not stored on the same rack to avoid total failure.
- Provides **fault tolerance** and **better latency** in case of node failures.

Replica Placement Strategy

- **Replication Factor:** Typically 3 (default in HDFS).
- **Placement Strategy:**
 - First replica on the **local DataNode**.
 - Second replica on another DataNode in the **same rack**.
 - Third replica on a **different rack** to ensure fault tolerance.

HDFS Write Operation

Step-by-step process:

- Client requests **metadata** from NameNode.
- NameNode responds with **block information** and replica locations.
- Client writes block **A** to DataNode 1.
- DataNode 1 copies the block to DataNode 2 (same rack via **rack switch**).
- DataNode 2 copies the block to DataNode 4 (different rack via **out-of-rack switch**).
- DataNodes send **write confirmation** to the NameNode.
- Process repeats for all blocks of the file.

HDFS Read Operation

Step-by-step process:

- Client requests **metadata** from NameNode.
- NameNode provides **DataNode locations** for required blocks.
- Client **directly interacts** with DataNodes for block retrieval.
- Data is read **in parallel** from multiple DataNodes.
- Blocks are **combined** into the original file by the client/application.

Java Interfaces in Big Data Processing

- Java interfaces help define data flow and interactions in big data systems.
- They enable modular, reusable, and loosely coupled components.

1. InputFormat Interface

- Defines how to read and split input data for MapReduce processing.
- Helps in creating **input splits** for distributed data processing.

2. OutputFormat Interface

- Specifies how to write output data after processing.
- Enables writing results to **files, databases, or other storage systems**.

3. Mapper & Reducer Interfaces

- **Mapper**: Converts input data into key-value pairs.
- **Reducer**: Aggregates and processes intermediate key-value results.
- Core components of **MapReduce programming**.



Java Interfaces in Big Data Processing

4. Input & Output Interfaces in Big Data Frameworks

- Used in **Apache Spark, Apache Flink**, etc.
- Define **data sources** (files, databases, streaming data) and **data sinks** (storage, messaging systems).

5. Connector Interfaces

- Enable integration with big data systems like **Apache Kafka, Cassandra, HBase**.
- Provide APIs to **transfer and process data efficiently**.

6. Custom Interfaces

- Allows defining application-specific data flow requirements.
- Useful for **communication, data structures, and specialized processing**.

Benefits of Using Java Interfaces in Big Data

- **Loose Coupling:** Easy to switch implementations without affecting functionality.
- **Modularity:** Promotes reusable components.
- **Maintainability & Extensibility:** Organizes and abstracts data flow for efficient system management.



Hadoop I/O

- Handles multi-terabyte datasets efficiently
- Ensures data integrity, compression, and serialization
- Key components: Checksum, CRC, Data Replication, Writable Format

Data Integrity & Compression

- **Data Integrity:**
 - Uses checksums & CRC for error detection & correction
 - DataNodes verify blocks before storage & processing
 - Periodic verification with Data Block Scanner
 - Replication mechanism for fault tolerance
- **Compression:**
 - Essential for reducing storage & speeding up data transfer
 - Common formats: gzip, bzip2, LZO, zip



Hadoop I/O

- **Serialization:**
 - Converts structured objects into byte streams for storage & transmission
 - Uses RPC for inter-process communication
 - Writable format optimizes MapReduce key-value pairs
- **Hadoop File Structures:**
 - **SequenceFile:** Stores binary key-value pairs
 - **MapFile:** Sorted SequenceFile for fast lookups

Hadoop File Data Structures

- **SequenceFile:**
 - Stores binary key-value pairs
 - Example: Key (timestamp) → Value (Writable data)
- **MapFile:**
 - Sorted version of SequenceFile
 - Provides an index for quick lookups
- **Interoperability:**
 - SequenceFile and MapFile can be converted between each other



Avro - A Powerful Serialization Framework

- **Data Serialization:**
 - Defines schemas in JSON format
 - Supports complex structures: records, arrays, maps, enums
- **Compact Binary Format:**
 - Efficient storage & fast data transmission
 - Uses compression & schema resolution
- **Schema Evolution:**
 - Backward & forward compatibility for updates

Avro in the Hadoop Ecosystem

- **Interoperability:**
 - Supports multiple programming languages
 - Ensures smooth data exchange across systems
- **Hadoop Integration:**
 - Works with HDFS, MapReduce, Spark, Hive, Kafka
 - Default serialization format for Hadoop MapReduce
- **Avro Data Files:**
 - Store multiple records with schemas
 - Support schema evolution for long-term data storage

In every lecture, Faculty may talk on any of the following:

- **DCS & GSFCU Portal:** Remind students to use DCS, talk about it's features and instruct all to visit GSFCU website
- **Discipline, Values and Ethics.**
- **GSFC University – My University:** Emphasize the importance of taking ownership of the University.
- **Respect & Courtesy:** Remind students to be respectful to peers, teachers, and staff. Encourage polite language and behavior.
- **Punctuality:** Emphasize the importance of arriving on time to class and completing assignments by deadlines.
- **Classroom Rules:** Reinforce basic classroom rules, such as keeping mobile phones on silent and maintaining a clean environment.
- **Safety:** Remind students about safety protocols, whether it's physical safety in the classroom or digital safety when using online resources.
- **Positive Attitude:** Encourage a positive mindset, cooperation and supporting one another.

GSFCU & Values

Respect & Courtesy



Dress
Appropriately



Respect
Personal Space



Be Mindful
of Your Words



Respect
Coworkers' Time



Keep Your
Workplace Clean



Socialise with
Your Peers



Use Your
Mobile Wisely



Take
Responsibility